

Parte 9 - Sicurezza, autenticazione e autorizzazione

Classificazione delle API e Sicurezza

I servizi web espongono le proprie funzionalità tramite interfacce (API). La sicurezza di queste API dipende fortemente dal loro scopo e dalla loro classificazione:

- **API Pubbliche:** sono accessibili liberamente a chiunque su internet. Il rischio maggiore è l'abuso massivo.
- **API Private:** sono esposte su Internet, ma destinate solo all'uso interno di applicazioni proprietarie (es. l'app mobile dell'azienda).
- **API Interne:** sono confinate all'interno di una rete privata (es. un cluster Kubernetes) e accessibili solo da microservizi autorizzati.

Vulnerabilità Comuni e Contromisure

Progettare un sistema sicuro significa prevenire una vasta gamma di attacchi. Di seguito, i principali e le relative soluzioni:

- **Denial of Service (DoS) e DDoS:** un attacco volto a esaurire le risorse del server inviando un volume enorme di richieste, rendendo il servizio inaccessibile agli utenti legittimi. Se l'attacco proviene da una singola sorgente (DoS), la difesa principale è il *Load-Balancing* (distribuire il carico su più server). Se l'attacco è distribuito da migliaia di macchine (DDoS), è necessaria una CDN (*Content Delivery Network*) distribuita geograficamente e sistemi di validazione dello *User-Agent* (es. captcha) per distinguere gli umani dai bot. Per le API pubbliche, è fondamentale implementare *Usage-Quota* (es. rate-limiting: max 10 richieste al secondo per utente).
- **Man in the Middle (MITM):** l'attaccante intercetta la comunicazione tra l'utente e il server (es. su una rete Wi-Fi pubblica compromessa) per rubare dati sensibili o credenziali. La contromisura standard è forzare la cifratura del traffico imponendo il protocollo HTTPS (HTTPS-Only Standard o HSTS).
- **Cross-Site Request Forgery (CSRF):** sfrutta l'autenticazione attiva di un utente. Se un utente è loggato sul sito della banca e visita un sito malevolo, quest'ultimo può forzare il browser a eseguire chiamate "ombra" verso la banca a nome dell'utente. Si previene generando un **Token anti-CSRF** crittografico nascosto in ogni form HTML legittimo; il server rifiuterà ogni richiesta priva di questo token (una funzionalità gestita automaticamente da ASP.Net Core).
- **SQL-Injection:** l'attaccante inserisce codice SQL malvolo all'interno degli input utente (es. nei campi di login) per manipolare le query ed estrarre dati o bypassare le difese. La soluzione è separare rigorosamente i dati dal codice utilizzando *Query Parametrizzate* (variabili) o un ORM (Object-Relational Mapping) come Entity Framework, che gestisce questa separazione in automatico.
- **Cross-site Scripting (XSS):** l'attaccante inietta codice eseguibile (solitamente JavaScript) all'interno di una pagina web visualizzata da altri utenti (es. inserendo script in un campo "biografia" di un profilo). La difesa consiste nell'*escape* (o sanitizzazione) dei dati mostrati a schermo (ASP.NET Core Razor lo fa di default) e nell'implementazione di direttive CSP (Content Security Policy) a livello di browser.
- **Abuso di API Private/Interne:** per impedire che client non ufficiali o script esterni utilizzino le API, si configurano le direttive **CORS (Cross-Origin Resource Sharing)** per indicare al browser di accettare richieste

solo da domini autorizzati, abbinando sempre un robusto sistema di autenticazione basato su token e analisi dei log.

Gestione Sicura delle Password

Salvare le password degli utenti nel database in chiaro (Clear-text) è una grave negligenza; in caso di furto del database, tutte le credenziali sarebbero immediatamente compromesse. Le password devono essere processate:

1. **Hash:** si trasforma la password in una stringa indecifrabile tramite una funzione unidirezionale. Tuttavia, password comuni possono essere scoperte usando tabelle pre-calcolate (*Rainbow-tables*) o *Dictionary-attacks*.
2. **Hash con Salt:** si aggiunge una stringa casuale (il Salt) alla password prima di farne l'hash. Questo rende inefficaci le Rainbow-tables, ma la funzione rimane esposta alla forza bruta.
3. **Hash con Salt e Key-Derivation (lo standard):** si utilizza un algoritmo (come PBKDF2, Argon2 o BCrypt) progettato specificatamente per essere "lento" o per consumare molta memoria. Questo approccio rende i tentativi di forza bruta matematicamente proibitivi e troppo lenti per essere efficaci.

Autenticazione e Autorizzazione

Sebbene spesso usati come sinonimi, i due concetti sono distinti: l'**autenticazione** risponde alla domanda "Chi sei?" (identità dell'utente), mentre l'**autorizzazione** risponde a "cosa puoi fare?" (i permessi sulle risorse).

Esistono diversi meccanismi per gestire l'identità in una comunicazione stateless come HTTP:

- **Cookie:** Il server genera un file firmato contenente l'identità dell'utente, che il browser invia automaticamente a ogni richiesta successiva. È semplice, ma vincolato al singolo dominio e non è adatto per autenticazioni tra servizi o domini differenti.
- **Bearer Token (Token al Portatore):** Un token crittografico rilasciato al login che il client deve allegare esplicitamente (di solito nell'header HTTP `Authorization: Bearer <token>`) per identificarsi.
- **JSON Web Token (JWT):** È un Bearer Token standardizzato e strutturato in tre parti separate da un punto (`Intestazione.Payload.Firma`), tutte codificate in Base64. Il *Payload* contiene informazioni (claims) leggibili come l'ID utente o la data di scadenza. La *Firma* garantisce che il contenuto non sia stato manomesso.

Il Protocollo OpenID Connect (OIDC) e OAuth 2.0

Per le architetture a microservizi distribuite, si utilizza lo standard **OAuth 2.0**, su cui si basa **OpenID Connect (OIDC)**, il livello specifico per l'autenticazione.

OIDC decentralizza la gestione dell'identità isolando gli attori coinvolti:

1. Il **Client** (l'app web) desidera accedere alle risorse. Invece di chiedere la password all'utente, lo reindirizza all'**Authorization Server** (il provider di identità).
2. L'utente inserisce le credenziali direttamente sull'Authorization Server. Se il login ha successo, l'utente viene reindirizzato indietro al Client con un *Token di Autorizzazione*.
3. Il Client contatta il **Token Service** e scambia il token precedente con un vero e proprio Bearer Token (spesso un JWT).
4. Infine, il Client invia questo Bearer Token al **Resource Server** (le API), il quale, previa validazione del token, restituisce i dati richiesti.

Questo flusso garantisce un'altissima sicurezza: l'applicazione client non tocca mai le password degli utenti e la gestione dei token è interamente delegata al browser (lo user-agent) e al server di autenticazione dedicato. In ambiente .NET, l'implementazione open-source di riferimento per OIDC è **OpenIddict**.